МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 2 по курсу «Операционные системы и архитектура компьютера»

Выполнил: М. С. Столповский
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 22.10.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: приобретение практических навыков в управлении процессами в ОС и обеспечении обмена данными между процессами посредством каналов.

Задание: составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем (выбрана Linux). В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант 22. Родительский процесс создаёт два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла на запись для child1, второй строкой – аналогично для child2. Родительский и дочерний процессы должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или pipe2 в зависимости от правила фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2. Дочерние процессы инвертируют строки и пишут результаты своей работы в стандартный вывод.

Метод решения

Решение состоит из двух программ: `parent` (родительский процесс) и `child` (дочерний процесс). Во время работы существует три процесса – `parent` и два экземпляра `child`, которые в логах называются child1 и child2. Схема их взаимодействия показана на рисунке 1.

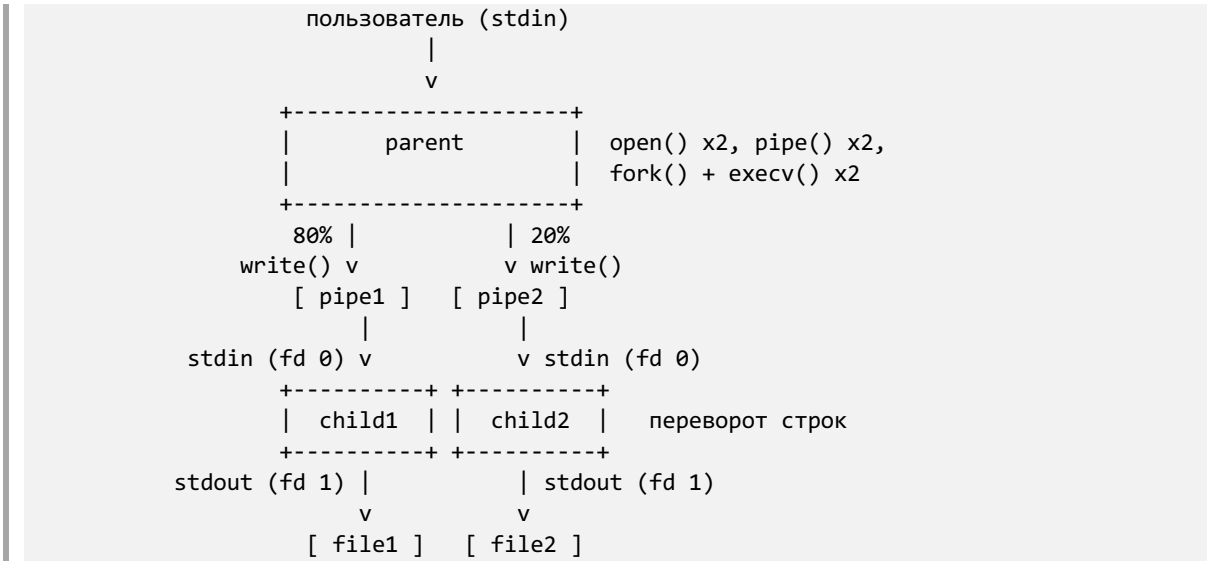


Рисунок 1 – Схема взаимодействия процессов

Родительский процесс сначала читает из стандартного ввода два имени файлов и открывает оба файла на запись (флаги `O_WRONLY | O_CREAT | O_TRUNC`). Если оба имени указывают на один и тот же файл (это проверяется через `fstat` по номеру устройства и inode), программа завершается с ошибкой. Затем создаются два канала и дважды вызывается `fork`. В каждом дочернем процессе до вызова `execv` стандартный ввод перенаправляется на конец своего канала для чтения, а стандартный вывод – на свой файл (`dup2`). Сама программа `child` ничего не знает о каналах и файлах: она читает строки из дескриптора 0, переворачивает их и пишет в дескриптор 1.

После запуска детей родитель читает строки пользователя и для каждой строки выбирает канал: если `rand() % 100 < 80`, строка уходит в pipe1, иначе в pipe2. Генератор инициализируется через `srand` значением `time(NULL) ^ getpid()`, а для повторяемых запусков можно передать зерно ключом `--seed`. Когда ввод заканчивается (EOF, Ctrl+D), родитель закрывает концы каналов для записи. После этого `read` в дочерних процессах возвращает 0, они дописывают оставшиеся строки и завершаются, а родитель забирает их статус вызовом `waitpid`, поэтому зомби-процессы не остаются.

Отдельно пришлось следить за закрытием дескрипторов. После `fork` ребёнок получает копию всех открытых дескрипторов родителя, в том числе концов обоих каналов. Если, например, в child2 останется открытой копия конца для записи pipe1, то child1 никогда не получит EOF и программа зависнет. Поэтому каждый дочерний процесс после `dup2` закрывает все шесть дескрипторов каналов и файлов, а родитель после запуска детей закрывает концы для чтения и файлы, оставляя себе только концы для записи.

Канал передаёт поток байт без границ сообщений: один вызов `read` может вернуть часть строки или сразу несколько строк. Поэтому строки передаются с символом перевода строки на конце, а получатель сам накапливает данные в буфере и делит их на строки. Буфер растёт через `realloc`, так что длина строки ограничена только памятью. Запись выполняется в цикле, потому что `write` может записать меньше байт, чем было запрошено.

Если дочерний процесс неожиданно завершится, запись в его канал по умолчанию приведёт к сигналу SIGPIPE, который завершит и родителя. Чтобы этого не произошло, родитель игнорирует SIGPIPE: тогда `write` возвращает ошибку `EPIPE`, родитель закрывает этот канал, считает строки для него потерянными и продолжает работу со вторым ребёнком. Строки переворачиваются посимвольно с учётом кодировки UTF-8 (буква кириллицы занимает два байта), иначе русский текст превращался бы в набор некорректных байтов.

При подготовке использовались страницы руководства man (fork(2), execve(2), pipe(2), pipe(7), dup2(2), waitpid(2), signal(7)), материалы лекций курса и книга Э. Таненбаума «Операционные системы. Разработка и реализация».

Описание программы

Программа написана на языке Си и собирается утилитой make компилятором gcc. Для работы с ОС используются только системные вызовы POSIX. Отладка и тестирование проводились в ОС Ubuntu 24.04 LTS (контейнер Docker).

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
`parent.c`	родительский процесс: читает имена файлов, открывает файлы, создаёт каналы, запускает два дочерних процесса, распределяет строки между каналами и ожидает завершения детей
`child.c`	дочерний процесс: читает строки из stdin, переворачивает их и пишет в stdout

Файл	Назначение
utils.h, utils.c	общий код: запись данных целиком, построчное чтение из дескриптора, переворот строки в UTF-8
Makefile	сборка программ parent и child в каталог build/

Основные типы данных и функции

LineReader – структура для построчного чтения из дескриптора: номер дескриптора, динамический буфер, границы непрочитанных данных и признак конца ввода. Функция lr_next(LineReader *r, char **line, size_t *len) возвращает LR_LINE (прочитана строка без символа перевода строки), LR_EOF (данных больше нет) или LR_ERROR (ошибка read или нехватка памяти). Указатель на строку указывает во внутренний буфер и действителен до следующего вызова. Функции lr_init и lr_free инициализируют структуру и освобождают буфер.

int write_all(int fd, const char *buf, size_t len) – записывает данные целиком, повторяя write, пока не будут записаны все байты. Возвращает 0 при успехе и −1 при ошибке (причина остаётся в errno).

void reverse_utf8(const char *src, size_t len, char *dst) – записывает в dst строку, перевёрнутую посимвольно. Длина каждого символа определяется по старшим битам его первого байта, некорректные последовательности переворачиваются побайтно.

pid_t spawn_child(...) – создаёт дочерний процесс: вызывает fork, в ребёнке перенаправляет stdin и stdout через dup2, закрывает все лишние дескрипторы и запускает программу child через execv. В родителе возвращает PID ребёнка или −1.

int wait_child(const char *name, pid_t pid) – ждёт завершения ребёнка через waitpid, выводит код завершения или номер сигнала, которым он был убит. Возвращает 0, если ребёнок завершился с кодом 0.

void close_fd(int *fd) – закрывает дескриптор, если он открыт, и записывает в переменную −1. За счёт этого все ресурсы можно освободить в одном месте (метка out в main) независимо от того, на каком шаге произошла ошибка.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
pid_t fork(void)	создание дочерних процессов child1 и child2
int execv(const char *path, char *const argv[])	замена образа дочернего процесса программой child
pid_t waitpid(pid_t pid, int *status, int options)	ожидание завершения дочерних процессов и получение их статуса
int pipe(int fd[2])	создание каналов pipe1 и pipe2
int dup2(int oldfd, int newfd)	перенаправление stdin ребёнка на канал, а stdout – на файл
int open(const char *path, int flags, mode_t mode)	открытие (создание) выходных файлов

Системный вызов	Назначение в программе
`int close(int fd)`	закрытие неиспользуемых концов каналов и дескрипторов файлов
`ssize_t read(int fd, void *buf, size_t n)`	чтение строк пользователя (parent) и строк из канала (child)
`ssize_t write(int fd, const void *buf, size_t n)`	передача строк в канал, запись результата в файл, вывод журнала в stderr
`int fstat(int fd, struct stat *st)`	проверка, что child1 и child2 пишут в разные файлы
`sighandler_t signal(int sig, sighandler_t h)`	игнорирование SIGPIPE в родителе и восстановление обработчика по умолчанию в детях
`int isatty(int fd)`	проверка, вводятся ли данные с терминала (для подсказок)
`pid_t getpid(void)`, `pid_t getppid(void)`	получение PID для журнала
`void _exit(int status)`	завершение ребёнка при ошибке `execv` без сброса буферов stdio, унаследованных от родителя

На архитектуре aarch64 часть этих функций glibc реализует через другие системные вызовы, поэтому в выводе strace вместо `fork`, `pipe`, `dup2` и `open` видны `clone`, `pipe2`, `dup3` и `openat`. Функции `execv`, `waitpid` и `signal` соответствуют вызовам `execve`, `wait4` и `rt_sigaction`, а `isatty` выполняется через `ioctl`.

Ключевые фрагменты кода

Создание дочернего процесса. После `fork` функция сразу возвращает управление родителю, а в ребёнке выполняется перенаправление ввода-вывода, закрытие всех дескрипторов каналов и файлов и запуск программы `child`.

Листинг 1 – Запуск дочернего процесса (parent.c)

```
static pid_t spawn_child(const char *path, char *name, int pipes[2][2], int files[2],
int i) {
    pid_t pid = fork();
    if (pid != 0) {
        return pid;
    }

    if (dup2(pipes[i][0], STDIN_FILENO) == -1 || dup2(files[i], STDOUT_FILENO) == -1) {
        perror("dup2");
        _exit(127);
    }
    for (int j = 0; j < 2; ++j) {
        close_fd(&pipes[j][0]);
        close_fd(&pipes[j][1]);
        close_fd(&files[j]);
    }
    signal(SIGPIPE, SIG_DFL);

    char *const args[] = {"child", name, NULL};
```

```
    execv(path, args);
    fprintf(stderr, "[%s pid=%ld] execv %s: %s\n", name, (long)getpid(), path,
strerror(errno));
    _exit(127);
}
```

Создание каналов и запуск обоих детей. После запуска родитель закрывает концы каналов для чтения и файлы: они нужны только детям.

Листинг 2 – Создание каналов и дочерних процессов (parent.c)

```
for (int i = 0; i < 2; ++i) {
    if (pipe(pipes[i]) == -1) {
        perror("pipe");
        goto out;
    }
}

for (int i = 0; i < 2; ++i) {
    pids[i] = spawn_child(path, names[i], pipes, files, i);
    if (pids[i] == -1) {
        perror("fork");
        goto out;
    }
    fprintf(stderr, "[parent pid=%ld] started %s pid=%ld\n", (long)getpid(), names[i],
(long)pids[i]);
}

for (int i = 0; i < 2; ++i) {
    close_fd(&pipes[i][0]);
    close_fd(&files[i]);
}
```

Распределение строк между каналами. При ошибке `EPIPE` (ребёнок перестал читать) канал закрывается, а строки, которые должны были в него попасть, считаются потерянными.

Листинг 3 – Распределение строк с вероятностью 80/20 (parent.c)

```
srand(seed);
long lineno = 0;
while ((st = lr_next(&in, &line, &len)) == LR_LINE) {
    ++lineno;
    int i = (rand() % 100 < 80) ? 0 : 1;

    if (pipes[i][1] == -1) {
        ++lost;
        fprintf(stderr, "[parent pid=%ld] line %ld -> pipe%d: pipe is closed, line
lost\n",
                (long)getpid(), lineno, i + 1);
        continue;
    }
    if (write_all(pipes[i][1], line, len) == -1 || write_all(pipes[i][1], "\n", 1) == -
1) {
        if (errno == EPIPE) {
            fprintf(stderr, "[parent pid=%ld] %s stopped reading pipe%d (EPIPE)\n",
                    (long)getpid(), names[i], i + 1);
        } else {
            perror("write");
        }
```

```
        close_fd(&pipes[i][1]);
        ++lost;
        continue;
    }
    ++sent[i];
    fprintf(stderr, "[parent pid=%ld] line %ld -> pipe%d (%s)\n", (long)getpid(),
lineno, i + 1, names[i]);
}
```

Освобождение ресурсов. Сначала закрываются все дескрипторы, в том числе концы каналов для записи (дети получают EOF), и только потом выполняется ожидание детей. Если закрыть каналы после `waitpid`, процессы будут ждать друг друга вечно.

Листинг 4 – Завершение работы родителя (parent.c)

```
out:
    for (int i = 0; i < 2; ++i) {
        close_fd(&pipes[i][0]);
        close_fd(&pipes[i][1]);
        close_fd(&files[i]);
    }
    for (int i = 0; i < 2; ++i) {
        if (pids[i] > 0 && wait_child(names[i], pids[i]) == -1) {
            rc = 1;
        }
    }
    if (pids[0] > 0 && pids[1] > 0) {
        long total = sent[0] + sent[1];
        fprintf(stderr, "[parent pid=%ld] done: pipe1 %ld (%.1f%%), pipe2 %ld (%.1f%%),
lost %ld\n",
                (long)getpid(), sent[0], total ? 100.0 * sent[0] / total : 0.0,
                sent[1], total ? 100.0 * sent[1] / total : 0.0, lost);
    }
    lr_free(&in);
```

Основной цикл дочернего процесса.

Листинг 5 – Обработка строк в дочернем процессе (child.c)

```
for (;;) {
    LineStatus st = lr_next(&in, &line, &len);
    if (st == LR_EOF) {
        break;
    }
    if (st == LR_ERROR) {
        perror("child: read");
        rc = 1;
        break;
    }

    char *out = malloc(len + 1);
    if (out == NULL) {
        perror("child: malloc");
        rc = 1;
        break;
    }
    reverse_utf8(line, len, out);
    out[len] = '\n';
    int write_ans = write_all(STDOUT_FILENO, out, len + 1);
    free(out);
```

```
    if (write_ans == -1) {
        perror("child: write");
        rc = 1;
        break;
    }
    ++count;
    fprintf(stderr, "[%s pid=%ld] line %ld reversed\n", name, pid, count);
}
```

Результаты

Программа собирается командой `make` в каталоге src и запускается как `./build/parent`. Ниже приведён запуск с зерном 14: на вход подаются два имени файлов и восемь строк. В журнале видны PID родителя и детей (у детей PPID совпадает с PID родителя) и то, что дети работают одновременно: их сообщения перемешаны.

```
$ ./build/parent --seed 14
> out1.txt
> out2.txt
> hello world
> привет мир
> операционные системы
> line four
> пятая строка
> six
> seven
> восемь

[parent pid=97] started, child program: /labs/lab2/src/build/child, seed=14
[parent pid=97] started child1 pid=98
[parent pid=97] started child2 pid=99
[parent pid=97] line 1 -> pipe2 (child2)
[parent pid=97] line 2 -> pipe1 (child1)
[parent pid=97] line 3 -> pipe1 (child1)
[parent pid=97] line 4 -> pipe1 (child1)
[parent pid=97] line 5 -> pipe1 (child1)
[parent pid=97] line 6 -> pipe2 (child2)
[parent pid=97] line 7 -> pipe1 (child1)
[parent pid=97] line 8 -> pipe1 (child1)
[child1 pid=98 ppid=97] started
[child2 pid=99 ppid=97] started
[child1 pid=98] line 1 reversed
[child1 pid=98] line 2 reversed
[child2 pid=99] line 1 reversed
[child2 pid=99] line 2 reversed
[child1 pid=98] line 3 reversed
[child1 pid=98] line 4 reversed
[child2 pid=99] EOF on stdin, 2 line(s) processed, exiting
[child1 pid=98] line 5 reversed
[child1 pid=98] line 6 reversed
[child1 pid=98] EOF on stdin, 6 line(s) processed, exiting
[parent pid=97] child1 pid=98 exited with code 0
[parent pid=97] child2 pid=99 exited with code 0
[parent pid=97] done: pipe1 6 (75.0%), pipe2 2 (25.0%), lost 0
$ cat out1.txt
рим тевирп
ыметсис еынноицарепо
ruof enil
```

```
акортс яатяп
neves
ьмесов
$ cat out2.txt
dlrow olleh
xis
```

Шесть строк попали в pipe1 и две в pipe2, в файлах лежат перевёрнутые строки, в том числе русские. Чтобы проверить само распределение, на вход было подано 10 000 строк:

```
$ { echo a.txt; echo b.txt; seq 1 10000; } | ./build/parent 2>&1 | tail -1
[parent pid=103] done: pipe1 7977 (79.8%), pipe2 2023 (20.2%), lost 0
$ wc -l a.txt b.txt
 7977 a.txt
 2023 b.txt
10000 total
```

В pipe1 ушло 79,8% строк, в pipe2 – 20,2%, что соответствует заданной вероятности. Обработка ошибочных ситуаций (один файл под двумя именами, несуществующий каталог):

```
$ ./build/parent        # одинаковые файлы
[parent pid=110] started, child program: /labs/lab2/src/build/child, seed=1792528210
child1 and child2 must write to different files
$ ./build/parent        # каталога не существует
[parent pid=112] started, child program: /labs/lab2/src/build/child, seed=1792528204
open /nope/x.txt: No such file or directory
```

Обрыв канала: во время работы child1 был завершён командой `kill -9`. Родитель не упал от SIGPIPE, а получил `EPIPE`, закрыл pipe1 и продолжил отправлять строки child2. После завершения программы процессов parent и child не осталось, то есть зомби-процессов нет.

```
$ ./build/parent --seed 1     # во время работы: kill -9 <pid child1>
[parent pid=114] started, child program: /labs/lab2/src/build/child, seed=1
[parent pid=114] started child1 pid=116
[parent pid=114] started child2 pid=117
[parent pid=114] line 1 -> pipe2 (child2)
[child1 pid=116 ppid=114] started
[child2 pid=117 ppid=114] started
[child2 pid=117] line 1 reversed
[parent pid=114] line 2 -> pipe2 (child2)
[parent pid=114] child1 stopped reading pipe1 (EPIPE)
[child2 pid=117] line 2 reversed
[parent pid=114] line 4 -> pipe1: pipe is closed, line lost
[parent pid=114] line 5 -> pipe2 (child2)
[parent pid=114] line 6 -> pipe1: pipe is closed, line lost
[child2 pid=117] line 3 reversed
[child2 pid=117] EOF on stdin, 3 line(s) processed, exiting
[parent pid=114] child1 pid=116 killed by signal 9
[parent pid=114] child2 pid=117 exited with code 0
[parent pid=114] done: pipe1 0 (0.0%), pipe2 3 (100.0%), lost 3
$ ps -eo pid,stat,comm | grep -E "parent|child"
(процессов parent/child не осталось)
```

Кроме того, программа проверялась на строках длиной 1 МБ (больше размера буфера канала), на пустом вводе, на отсутствии программы `child` (дети завершаются с кодом

127), а также в сборке с AddressSanitizer и UndefinedBehaviorSanitizer – ошибок работы с памятью и утечек обнаружено не было.

Демонстрация работы strace

Программа была запущена командой `strace -ff -o tr ./build/parent --seed 14 < in.txt`, где in.txt содержит имена файлов s1.txt, s2.txt и три строки. Ключ -ff записывает трассу каждого процесса в отдельный файл, поэтому вызовы разных процессов не перемешиваются. Полный вывод приведён в приложении Б.

Родитель (PID 12) открывает файлы, создаёт каналы и запускает двух детей:

```
openat(AT_FDCWD, "s1.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
openat(AT_FDCWD, "s2.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
pipe2([5, 6], 0)                        = 0
pipe2([7, 8], 0)                        = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffff82f5bfd0) = 13
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffff82f5bfd0) = 14
```

Файлы получили дескрипторы 3 и 4, pipe1 – 5 (чтение) и 6 (запись), pipe2 – 7 и 8. Функция `fork` реализована через `clone`, возвращённые значения 13 и 14 – PID детей.

```
dup3(5, 0, 0)                           = 0
dup3(3, 1, 0)                           = 1
close(5)                                = 0
close(6)                                = 0
close(3)                                = 0
close(7)                                = 0
close(8)                                = 0
close(4)                                = 0
execve("/labs/lab2/src/build/child", ["child", "child1"], 0xffffc4dc77f8 /* 7 vars */) =
0
```

Это child1 (PID 13): `dup3(5, 0)` подключает pipe1 к stdin, `dup3(3, 1)` – файл s1.txt к stdout, затем закрываются все шесть исходных дескрипторов, и процесс заменяется программой `child`. В child2 всё так же, но с дескрипторами 7 и 4.

```
close(5)                                = 0
close(3)                                = 0
close(7)                                = 0
close(4)                                = 0
write(8, "hello", 5)                    = 5
write(8, "\n", 1)                       = 1
write(6, "\320\277\321\200\320\270\320\262\320\265\321\202", 12) = 12
write(6, "\n", 1)                       = 1
write(6, "world", 5)                    = 5
write(6, "\n", 1)                       = 1
read(0, "", 4096)                       = 0
close(6)                                = 0
close(8)                                = 0
wait4(13, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 13
wait4(14, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 14
```

Родитель закрывает ненужные ему дескрипторы 5, 3, 7, 4 и пишет строки в каналы: «hello» ушла в pipe2 (дескриптор 8), остальные две – в pipe1 (дескриптор 6). После

read, вернувшего 0 (конец ввода), закрываются концы для записи 6 и 8 и родитель ждёт детей через wait4: оба завершились с кодом 0.

```
read(0, "hello\n", 4096)                = 6
write(1, "olleh\n", 6)                  = 6
read(0, "", 4096)                       = 0
exit_group(0)                           = ?
```

Это child2 (PID 14): он прочитал из канала строку «hello», записал в файл «olleh», затем read вернул 0, так как все концы pipe2 для записи закрыты, и процесс завершился.

Выводы

В ходе работы была написана программа на Си, в которой родительский процесс создаёт два дочерних процесса и передаёт им строки через неименованные каналы, выбирая канал случайно с вероятностью 80/20. Дочерние процессы переворачивают строки и пишут результат в файлы, на которые родитель перенаправил их стандартный вывод. На практике были разобраны системные вызовы fork, execv, waitpid, pipe и dup2, наследование файловых дескрипторов и условия, при которых чтение из канала возвращает EOF. Правильность работы подтверждается содержимым выходных файлов, распределением строк на большом входе и трассировкой strace. Основная сложность была связана с закрытием неиспользуемых дескрипторов: пока хотя бы один процесс держит открытым конец канала для записи, читатель не получает EOF и программа зависает.

Приложение А. Исходный код программы

Листинг А.1 – Файл parent.c

```
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static int child_path(const char *argv0, char *buf, size_t size) {
    const char *slash = strrchr(argv0, '/');
    int n;
    if (slash == NULL) {
        n = snprintf(buf, size, "./child");
    } else {
        n = snprintf(buf, size, "%.*s/child", (int)(slash - argv0), argv0);
    }
    return (n < 0 || (size_t)n >= size) ? -1 : 0;
}

static void close_fd(int *fd) {
    if (*fd != -1 && close(*fd) == -1) {
```

```
        perror("close");
    }
    *fd = -1;
}

static pid_t spawn_child(const char *path, char *name, int pipes[2][2], int files[2],
int i) {
    pid_t pid = fork();
    if (pid != 0) {
        return pid;
    }

    if (dup2(pipes[i][0], STDIN_FILENO) == -1 || dup2(files[i], STDOUT_FILENO) == -1) {
        perror("dup2");
        _exit(127);
    }
    for (int j = 0; j < 2; ++j) {
        close_fd(&pipes[j][0]);
        close_fd(&pipes[j][1]);
        close_fd(&files[j]);
    }
    signal(SIGPIPE, SIG_DFL);

    char *const args[] = {"child", name, NULL};
    execv(path, args);
    fprintf(stderr, "[%s pid=%ld] execv %s: %s\n", name, (long)getpid(), path,
strerror(errno));
    _exit(127);
}

static int wait_child(const char *name, pid_t pid) {
    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid");
        return -1;
    }
    if (WIFEXITED(status)) {
        fprintf(stderr, "[parent pid=%ld] %s pid=%ld exited with code %d\n",
                (long)getpid(), name, (long)pid, WEXITSTATUS(status));
        return WEXITSTATUS(status) == 0 ? 0 : -1;
    }
    if (WIFSIGNALED(status)) {
        fprintf(stderr, "[parent pid=%ld] %s pid=%ld killed by signal %d\n",
                (long)getpid(), name, (long)pid, WTERMSIG(status));
    }
    return -1;
}

int main(int argc, char *argv[]) {
    unsigned seed = (unsigned)time(NULL) ^ (unsigned)getpid();
    if (argc == 3 && strcmp(argv[1], "--seed") == 0) {
        seed = (unsigned)strtoul(argv[2], NULL, 10);
    } else if (argc != 1) {
        fprintf(stderr, "usage: %s [--seed N]\n", argv[0]);
        return 2;
    }

    char path[PATH_MAX];
    if (child_path(argv[0], path, sizeof path) == -1) {
        fprintf(stderr, "path to child is too long\n");
```

```
        return 1;
    }
    fprintf(stderr, "[parent pid=%ld] started, child program: %s, seed=%u\n",
(long)getpid(), path, seed);

    if (signal(SIGPIPE, SIG_IGN) == SIG_ERR) {
        perror("signal");
        return 1;
    }

    char *names[2] = {"child1", "child2"};
    int files[2] = {-1, -1};
    int pipes[2][2] = {{-1, -1}, {-1, -1}};
    pid_t pids[2] = {-1, -1};
    long sent[2] = {0, 0};
    long lost = 0;
    int rc = 1;

    LineReader in;
    lr_init(&in, STDIN_FILENO);
    int interactive = isatty(STDIN_FILENO);
    char *line;
    size_t len;
    LineStatus st;

    for (int i = 0; i < 2; ++i) {
        if (interactive) {
            fprintf(stderr, "File name for %s: ", names[i]);
        }
        st = lr_next(&in, &line, &len);
        if (st == LR_ERROR) {
            perror("read");
            goto out;
        }
        if (st == LR_EOF) {
            fprintf(stderr, "expected two file names\n");
            goto out;
        }
        files[i] = open(line, O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (files[i] == -1) {
            fprintf(stderr, "open %s: %s\n", line, strerror(errno));
            goto out;
        }
    }

    struct stat info[2];
    for (int i = 0; i < 2; ++i) {
        if (fstat(files[i], &info[i]) == -1) {
            perror("fstat");
            goto out;
        }
    }
    if (info[0].st_dev == info[1].st_dev && info[0].st_ino == info[1].st_ino) {
        fprintf(stderr, "child1 and child2 must write to different files\n");
        goto out;
    }

    for (int i = 0; i < 2; ++i) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
```

```
            goto out;
        }
    }

    for (int i = 0; i < 2; ++i) {
        pids[i] = spawn_child(path, names[i], pipes, files, i);
        if (pids[i] == -1) {
            perror("fork");
            goto out;
        }
        fprintf(stderr, "[parent pid=%ld] started %s pid=%ld\n", (long)getpid(),
names[i], (long)pids[i]);
    }

    for (int i = 0; i < 2; ++i) {
        close_fd(&pipes[i][0]);
        close_fd(&files[i]);
    }

    srand(seed);
    long lineno = 0;
    while ((st = lr_next(&in, &line, &len)) == LR_LINE) {
        ++lineno;
        int i = (rand() % 100 < 80) ? 0 : 1;

        if (pipes[i][1] == -1) {
            ++lost;
            fprintf(stderr, "[parent pid=%ld] line %ld -> pipe%d: pipe is closed, line
lost\n",
                    (long)getpid(), lineno, i + 1);
            continue;
        }
        if (write_all(pipes[i][1], line, len) == -1 || write_all(pipes[i][1], "\n", 1)
== -1) {
            if (errno == EPIPE) {
                fprintf(stderr, "[parent pid=%ld] %s stopped reading pipe%d (EPIPE)\n",
                        (long)getpid(), names[i], i + 1);
            } else {
                perror("write");
            }
            close_fd(&pipes[i][1]);
            ++lost;
            continue;
        }
        ++sent[i];
        fprintf(stderr, "[parent pid=%ld] line %ld -> pipe%d (%s)\n", (long)getpid(),
lineno, i + 1, names[i]);
    }
    if (st == LR_ERROR) {
        perror("read");
    } else if (lost == 0) {
        rc = 0;
    }

out:
    for (int i = 0; i < 2; ++i) {
        close_fd(&pipes[i][0]);
        close_fd(&pipes[i][1]);
        close_fd(&files[i]);
    }
```

```
    for (int i = 0; i < 2; ++i) {
        if (pids[i] > 0 && wait_child(names[i], pids[i]) == -1) {
            rc = 1;
        }
    }
    if (pids[0] > 0 && pids[1] > 0) {
        long total = sent[0] + sent[1];
        fprintf(stderr, "[parent pid=%ld] done: pipe1 %ld (%.1f%%), pipe2 %ld (%.1f%%),
lost %ld\n",
                (long)getpid(), sent[0], total ? 100.0 * sent[0] / total : 0.0,
                sent[1], total ? 100.0 * sent[1] / total : 0.0, lost);
    }
    lr_free(&in);
    return rc;
}
```

Листинг А.2 – Файл child.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[]) {
    const char *name = argc > 1 ? argv[1] : "child";
    long pid = (long)getpid();

    fprintf(stderr, "[%s pid=%ld ppid=%ld] started\n", name, pid, (long)getppid());

    LineReader in;
    lr_init(&in, STDIN_FILENO);
    char *line;
    size_t len;
    long count = 0;
    int rc = 0;

    for (;;) {
        LineStatus st = lr_next(&in, &line, &len);
        if (st == LR_EOF) {
            break;
        }
        if (st == LR_ERROR) {
            perror("child: read");
            rc = 1;
            break;
        }

        char *out = malloc(len + 1);
        if (out == NULL) {
            perror("child: malloc");
            rc = 1;
            break;
        }
        reverse_utf8(line, len, out);
        out[len] = '\n';
        int write_ans = write_all(STDOUT_FILENO, out, len + 1);
        free(out);
        if (write_ans == -1) {
            perror("child: write");
```

```
            rc = 1;
            break;
        }
        ++count;
        fprintf(stderr, "[%s pid=%ld] line %ld reversed\n", name, pid, count);
    }

    if (rc == 0) {
        fprintf(stderr, "[%s pid=%ld] EOF on stdin, %ld line(s) processed, exiting\n",
name, pid, count);
    }
    lr_free(&in);
    return rc;
}
```

Листинг А.3 – Файл utils.h

```
#ifndef UTILS_H
#define UTILS_H

#include <stddef.h>

typedef enum { LR_LINE, LR_EOF, LR_ERROR } LineStatus;

typedef struct {
    int fd;
    char *buf;
    size_t start;
    size_t len;
    size_t cap;
    int eof;
} LineReader;

int write_all(int fd, const char *buf, size_t len);

void lr_init(LineReader *r, int fd);
LineStatus lr_next(LineReader *r, char **line, size_t *len);
void lr_free(LineReader *r);

void reverse_utf8(const char *src, size_t len, char *dst);

#endif
```

Листинг А.4 – Файл utils.c

```
#include "utils.h"

#include <errno.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define LR_CHUNK 4096

int write_all(int fd, const char *buf, size_t len) {
    while (len > 0) {
        ssize_t n = write(fd, buf, len);
        if (n == -1) {
            if (errno == EINTR) {
                continue;
```

```
            }
            return -1;
        }
        buf += n;
        len -= (size_t)n;
    }
    return 0;
}

void lr_init(LineReader *r, int fd) {
    r->fd = fd;
    r->buf = NULL;
    r->start = 0;
    r->len = 0;
    r->cap = 0;
    r->eof = 0;
}

void lr_free(LineReader *r) {
    free(r->buf);
    r->buf = NULL;
    r->start = 0;
    r->len = 0;
    r->cap = 0;
}

LineStatus lr_next(LineReader *r, char **line, size_t *len) {
    for (;;) {
        char *nl = NULL;
        if (r->len > r->start) {
            nl = memchr(r->buf + r->start, '\n', r->len - r->start);
        }
        if (nl != NULL) {
            *nl = '\0';
            *line = r->buf + r->start;
            *len = (size_t)(nl - *line);
            r->start += *len + 1;
            return LR_LINE;
        }
        if (r->eof) {
            if (r->start == r->len) {
                return LR_EOF;
            }
            r->buf[r->len] = '\0';
            *line = r->buf + r->start;
            *len = r->len - r->start;
            r->start = r->len;
            return LR_LINE;
        }

        if (r->start > 0) {
            memmove(r->buf, r->buf + r->start, r->len - r->start);
            r->len -= r->start;
            r->start = 0;
        }
        if (r->cap - r->len < LR_CHUNK + 1) {
            size_t new_cap = r->cap ? r->cap : LR_CHUNK + 1;
            while (new_cap - r->len < LR_CHUNK + 1) {
                new_cap *= 2;
            }
```

```
            char *p = realloc(r->buf, new_cap);
            if (p == NULL) {
                return LR_ERROR;
            }
            r->buf = p;
            r->cap = new_cap;
        }

        ssize_t n = read(r->fd, r->buf + r->len, LR_CHUNK);
        if (n == -1) {
            if (errno == EINTR) {
                continue;
            }
            return LR_ERROR;
        }
        if (n == 0) {
            r->eof = 1;
        } else {
            r->len += (size_t)n;
        }
    }
}

static size_t utf8_char_len(const char *s, size_t len, size_t i) {
    unsigned char c = (unsigned char)s[i];
    size_t n = 1;
    if ((c & 0xE0) == 0xC0) {
        n = 2;
    } else if ((c & 0xF0) == 0xE0) {
        n = 3;
    } else if ((c & 0xF8) == 0xF0) {
        n = 4;
    }
    if (i + n > len) {
        return 1;
    }
    for (size_t k = 1; k < n; ++k) {
        if (((unsigned char)s[i + k] & 0xC0) != 0x80) {
            return 1;
        }
    }
    return n;
}

void reverse_utf8(const char *src, size_t len, char *dst) {
    size_t i = 0;
    while (i < len) {
        size_t n = utf8_char_len(src, len, i);
        memcpy(dst + len - i - n, src + i, n);
        i += n;
    }
}
```

Листинг А.5 – Файл Makefile

```
CC     := gcc
CFLAGS := -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Wpedantic -O2 -g
BUILD  := build

COMMON := $(BUILD)/utils.o
```

```
.PHONY: all parent child clean

all: parent child

parent: $(BUILD)/parent
child:  $(BUILD)/child

$(BUILD)/parent: $(BUILD)/parent.o $(COMMON)
    $(CC) $(CFLAGS) $^ -o $@

$(BUILD)/child: $(BUILD)/child.o $(COMMON)
    $(CC) $(CFLAGS) $^ -o $@

$(BUILD)/%.o: %.c | $(BUILD)
    $(CC) $(CFLAGS) -MMD -MP -c $< -o $@

$(BUILD):
    mkdir -p $@

clean:
    rm -rf $(BUILD)

-include $(wildcard $(BUILD)/*.d)
```

Приложение Б. Полный вывод strace

Вывод команды `strace -ff -o tr ./build/parent --seed 14 < in.txt`: по одному листингу на каждый процесс.

Листинг Б.1 – Файл tr.12 (процесс parent)

```
execve("/labs/lab2/src/build/parent", ["/labs/lab2/src/build/parent", "--seed", "14"],
0xffffc894a4e8 /* 7 vars */) = 0
brk(NULL)                               = 0xaaaae0c9d000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xffff82f5b000
faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=9727, ...}) = 0
mmap(NULL, 9727, PROT_READ, MAP_PRIVATE, 3, 0) = 0xffff82f58000
close(3)                                = 0
openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\360\206\2\0\0\0\0\0"..., 832)
= 832
fstat(3, {st_mode=S_IFREG|0755, st_size=1722920, ...}) = 0
mmap(NULL, 1892240, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
0xffff82d54000
mmap(0xffff82d60000, 1826704, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0) = 0xffff82d60000
munmap(0xffff82d54000, 49152)           = 0
munmap(0xffff82f1e000, 16272)           = 0
mprotect(0xffff82efa000, 77824, PROT_NONE) = 0
mmap(0xffff82f0d000, 20480, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x19d000) = 0xffff82f0d000
mmap(0xffff82f12000, 49040, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -
1, 0) = 0xffff82f12000
close(3)                                = 0
set_tid_address(0xffff82f5bfd0)         = 12
```

```
set_robust_list(0xffff82f5bfe0, 24)     = 0
rseq(0xffff82f5c620, 0x20, 0, 0xd428bc00) = 0
mprotect(0xffff82f0d000, 12288, PROT_READ) = 0
mprotect(0xaaaabc82f000, 4096, PROT_READ) = 0
mprotect(0xffff82f60000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0xffff82f58000, 9727)            = 0
getpid()                                = 12
getpid()                                = 12
write(2, "[parent pid=12] started, child p"..., 76) = 76
rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[],
sa_flags=SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
ioctl(0, TCGETS, 0xffffc4dc6380)        = -1 ENOTTY (Inappropriate ioctl for device)
getrandom("\x2f\xd4\xa4\xa4\x0b\xe0\x98\x1d", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0xaaaae0c9d000
brk(0xaaaae0cbe000)                     = 0xaaaae0cbe000
read(0, "s1.txt\ns2.txt\nhello\n\320\277\321\200\320\270\320\262\320\265\321\202"...,
4096) = 39
openat(AT_FDCWD, "s1.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
openat(AT_FDCWD, "s2.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
fstat(3, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
fstat(4, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
pipe2([5, 6], 0)                        = 0
pipe2([7, 8], 0)                        = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffff82f5bfd0) = 13
getpid()                                = 12
write(2, "[parent pid=12] started child1 p"..., 38) = 38
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffff82f5bfd0) = 14
getpid()                                = 12
write(2, "[parent pid=12] started child2 p"..., 38) = 38
close(5)                                = 0
close(3)                                = 0
close(7)                                = 0
close(4)                                = 0
write(8, "hello", 5)                    = 5
write(8, "\n", 1)                       = 1
getpid()                                = 12
write(2, "[parent pid=12] line 1 -> pipe2 "..., 41) = 41
write(6, "\320\277\321\200\320\270\320\262\320\265\321\202", 12) = 12
write(6, "\n", 1)                       = 1
getpid()                                = 12
write(2, "[parent pid=12] line 2 -> pipe1 "..., 41) = 41
write(6, "world", 5)                    = 5
write(6, "\n", 1)                       = 1
getpid()                                = 12
write(2, "[parent pid=12] line 3 -> pipe1 "..., 41) = 41
read(0, "", 4096)                       = 0
close(6)                                = 0
close(8)                                = 0
wait4(13, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 13
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=13, si_uid=0, si_status=0,
si_utime=0, si_stime=0} ---
getpid()                                = 12
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=14, si_uid=0, si_status=0,
si_utime=0, si_stime=0} ---
write(2, "[parent pid=12] child1 pid=13 ex"..., 49) = 49
wait4(14, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 14
```

```
getpid()                                = 12
write(2, "[parent pid=12] child2 pid=14 ex"..., 49) = 49
getpid()                                = 12
write(2, "[parent pid=12] done: pipe1 2 (6"..., 63) = 63
exit_group(0)                           = ?
+++ exited with 0 +++
```

Листинг Б.2 – Файл tr.13 (процесс child1)

```
set_robust_list(0xffff82f5bfe0, 24)     = 0
dup3(5, 0, 0)                           = 0
dup3(3, 1, 0)                           = 1
close(5)                                = 0
close(6)                                = 0
close(3)                                = 0
close(7)                                = 0
close(8)                                = 0
close(4)                                = 0
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[],
sa_flags=SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000}, {sa_handler=SIG_IGN,
sa_mask=[], sa_flags=SA_NODEFER|SA_RESETHAND}, 8) = 0
execve("/labs/lab2/src/build/child", ["child", "child1"], 0xffffc4dc77f8 /* 7 vars */) =
0
brk(NULL)                               = 0xaaab23edd000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xffff95619000
faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=9727, ...}) = 0
mmap(NULL, 9727, PROT_READ, MAP_PRIVATE, 3, 0) = 0xffff95616000
close(3)                                = 0
openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\360\206\2\0\0\0\0"..., 832)
= 832
fstat(3, {st_mode=S_IFREG|0755, st_size=1722920, ...}) = 0
mmap(NULL, 1892240, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
0xffff95412000
mmap(0xffff95420000, 1826704, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0) = 0xffff95420000
munmap(0xffff95412000, 57344)           = 0
munmap(0xffff955de000, 8080)            = 0
mprotect(0xffff955ba000, 77824, PROT_NONE) = 0
mmap(0xffff955cd000, 20480, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x19d000) = 0xffff955cd000
mmap(0xffff955d2000, 49040, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -
1, 0) = 0xffff955d2000
close(3)                                = 0
set_tid_address(0xffff95619fb0)         = 13
set_robust_list(0xffff95619fc0, 24)     = 0
rseq(0xffff9561a600, 0x20, 0, 0xd428bc00) = 0
mprotect(0xffff955cd000, 12288, PROT_READ) = 0
mprotect(0xaaaaea47f000, 4096, PROT_READ) = 0
mprotect(0xffff9561e000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0xffff95616000, 9727)            = 0
getpid()                                = 13
getppid()                               = 12
write(2, "[child1 pid=13 ppid=12] started\n", 32) = 32
getrandom("\xae\xc9\x3e\xc1\x14\x12\x25\x68", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0xaaab23edd000
```

```
brk(0xaaab23efe000)                     = 0xaaab23efe000
read(0, "\320\277\321\200\320\270\320\262\320\265\321\202\nworld\n", 4096) = 19
write(1, "\321\202\320\265\320\262\320\270\321\200\320\277\n", 13) = 13
write(2, "[child1 pid=13] line 1 reversed\n", 32) = 32
write(1, "dlrow\n", 6)                  = 6
write(2, "[child1 pid=13] line 2 reversed\n", 32) = 32
read(0, "", 4096)                       = 0
write(2, "[child1 pid=13] EOF on stdin, 2 "..., 59) = 59
exit_group(0)                           = ?
+++ exited with 0 +++
```

Листинг Б.3 – Файл tr.14 (процесс child2)

```
set_robust_list(0xffff82f5bfe0, 24)     = 0
dup3(7, 0, 0)                           = 0
dup3(4, 1, 0)                           = 1
close(5)                                = 0
close(6)                                = 0
close(3)                                = 0
close(7)                                = 0
close(8)                                = 0
close(4)                                = 0
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[],
sa_flags=SA_INTERRUPT|SA_NODEFER|SA_RESETHAND|0xffffffff00000000}, {sa_handler=SIG_IGN,
sa_mask=[], sa_flags=SA_NODEFER|SA_RESETHAND}, 8) = 0
execve("/labs/lab2/src/build/child", ["child", "child2"], 0xffffc4dc77f8 /* 7 vars */) =
0
brk(NULL)                               = 0xaaaae01fa000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xffffbd69a000
faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=9727, ...}) = 0
mmap(NULL, 9727, PROT_READ, MAP_PRIVATE, 3, 0) = 0xffffbd697000
close(3)                                = 0
openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\360\206\2\0\0\0\0"..., 832)
= 832
fstat(3, {st_mode=S_IFREG|0755, st_size=1722920, ...}) = 0
mmap(NULL, 1892240, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
0xffffbd493000
mmap(0xffffbd4a0000, 1826704, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0) = 0xffffbd4a0000
munmap(0xffffbd493000, 53248)           = 0
munmap(0xffffbd65e000, 12176)           = 0
mprotect(0xffffbd63a000, 77824, PROT_NONE) = 0
mmap(0xffffbd64d000, 20480, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x19d000) = 0xffffbd64d000
mmap(0xffffbd652000, 49040, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -
1, 0) = 0xffffbd652000
close(3)                                = 0
set_tid_address(0xffffbd69afb0)         = 14
set_robust_list(0xffffbd69afc0, 24)     = 0
rseq(0xffffbd69b600, 0x20, 0, 0xd428bc00) = 0
mprotect(0xffffbd64d000, 12288, PROT_READ) = 0
mprotect(0xaaaac51df000, 4096, PROT_READ) = 0
mprotect(0xffffbd69f000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0xffffbd697000, 9727)            = 0
getpid()                                = 14
```

```
getppid()                               = 12
write(2, "[child2 pid=14 ppid=12] started\n", 32) = 32
getrandom("\x5e\xb1\x50\xa0\x3e\x34\x27\xee", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0xaaaae01fa000
brk(0xaaaae021b000)                     = 0xaaaae021b000
read(0, "hello\n", 4096)                = 6
write(1, "olleh\n", 6)                  = 6
write(2, "[child2 pid=14] line 1 reversed\n", 32) = 32
read(0, "", 4096)                       = 0
write(2, "[child2 pid=14] EOF on stdin, 1 "..., 59) = 59
exit_group(0)                           = ?
+++ exited with 0 +++
```